

操作系统实验指导书

实验一：进程管理

北京邮电大学 计算机学院

目录

一、实验目的	1
二、实验要求及内容	1
2.1 读者-写者 实验.....	1
2.2 生产者-消费者 实验.....	1
2.3 实验相关的 API 介绍.....	2
三、实验形式	4
四、实验设备环境	4
五、实验步骤	4
5.1 读者-写者 实验原理及步骤.....	5
5.2 读者-写者 编译及运行方法.....	8
5.3 读者-写者 实验结果说明.....	9
5.4 生产者-消费者 实验原理及步骤.....	10
5.5 生产者-消费者 编译及运行方法	10
5.6 生产者-消费者 实验结果说明.....	10
六、实验报告书写要求.....	10

一、实验目的

本实验旨在让学生动手设计一个进程同步互斥的实验，更深刻的理解进程协作机制。

openEuler 提供了互斥量(pthread_mutex_t)、信号量(sem_t)等同步对象和相应的系统调用，用于线程的互斥与同步。学生可通过对读者写者问题的调试或者生产者消费者问题的调试（选做其一即可），了解 openEuler 中的同步、互斥机制。

二、实验要求及内容

2.1 读者-写者 实验

利用 openEuler 信号量机制，实现读者写者问题的模拟。

在 openEuler 环境下，创建一个包含 n 个线程的控制进程。用这 n 个线程来表示 n 个读者或写者。每个线程按相应测试数据文件的要求，进行读写操作。请用信号量机制分别实现读者优先和写者优先的读者-写者问题。

读者-写者问题的读写操作限制：

- 1) 写-写互斥；
- 2) 读-写互斥；
- 3) 读-读允许；

读者优先的附加限制：如果一个读者申请进行读操作时已有另一读者正在进行读操作，则该读者可直接开始读操作。

写者优先的附加限制：如果一个读者申请进行读操作时已有另一写者在等待访问共享资源，则该读者必须等到没有写者处于等待状态后才能开始读操作。

运行结果显示要求：要求在每个线程创建、发出读写操作申请、开始读写操作和结束读写操作时分别显示一行提示信息，以确信所有处理都遵守相应的读写操作限制。

2.2 生产者-消费者 实验

利用 openEuler 信号量机制，实现生产者消费者问题的模拟。

在 openEuler 环境下，设计一个程序来解决有限缓冲问题，使用三个信号量：empty_sem（以记录有多少空位）、full_sem（以记录有多少满位）以及 mutex（二进制信号量或互斥信号量，以保护对缓冲插入与删除的操作）。对于本实验，empty_sem 与 full_sem 将采用标准计数信号量，而 mutex 将采用二进制信号量。生产者与消费者作为独立线程，在 empty_sem、full_sem、mutex 的同步前提下，

对缓冲进行插入与删除。

2.3 实验相关的 API 介绍

1. 线程创建

使用 `pthread_create()` 函数。

```
int pthread_create(pthread_t *thread, const pthread_attr_t* attr, void *(*start_routine)
(void *), void* arg);
```

参数：

第一个参数：指向线程标示符 `pthread_t` 的指针

第二个参数：设置线程的属性

第三个参数：线程运行函数的起始地址

第四个参数：运行函数的参数

2. 等待线程结束

等待函数 `pthread_join()` 用来等待一个线程的结束。

```
int pthread_join(pthread_t thread, void **retval);
```

参数：

`thread`: 线程标识符，即线程 ID，标识唯一线程。

`retval`: 用户定义的指针，用来存储被等待线程的返回值。

返回值：0 代表成功。失败，返回的则是错误号。

3. 退出

`pthread_exit()` 函数使线程退出并返回一个空指针类型的值，该值可以由 `pthread_join()` 函数来获取。

4. sleep

函数原型：

```
unsigned int sleep (unsigned int seconds);
```

5. 初始化互斥锁

6. 销毁互斥锁

函数 `pthread_mutex_destroy()` 负责销毁互斥锁。

```
int pthread_mutex_destroy(pthread_mutex_t *mutex);
```

`mutex` 指向要销毁的互斥锁的指针。

互斥锁销毁函数在执行成功后返回 0，否则返回错误码。

7. 锁定

函数 `pthread_mutex_lock()` 负责锁定互斥锁。

当返回时，该互斥锁已被锁定。调用线程是该互斥锁的属主。如果该互斥锁已被另一个线程锁定和拥有，则调用线程将阻塞，直到该互斥锁变为可用为止。

8. 解锁

`pthread_mutex_unlock()` 与 `pthread_mutex_lock()` 成对存在，负责释放互斥锁。

9. 创建信号量

`sem_init()` 函数用于创建信号量，其原型如下：

```
int sem_init(sem_t *sem, int pshared, unsigned int value);
```

该函数初始化由 `sem` 指向的信号对象，并给它一个初始的整数值 `value`。

`pshared` 控制信号量的类型，值为 0 代表该信号量用于多线程间的同步，值如果大于 0 表示可以共享，用于多个相关进程间的同步。

10. wait

`sem_wait()` 是一个阻塞的函数，测试所指定信号量的值，它的操作是原子的。

函数原型：

```
int sem_wait(sem_t *sem);
```

若 `sem value > 0`，则该信号量值减去 1 并立即返回。

若 `sem value = 0`，则阻塞直到 `sem value > 0`，此时立即减去 1，然后返回。

11. signal

`sem_post()` 把指定的信号量 `sem` 的值加 1，唤醒正在等待该信号量的任意线程。

函数原型：

```
int sem_post(sem_t *sem);
```

12. 清理

`sem_destroy()` 函数用于对用完的信号量的清理。函数原型：

```
int sem_destroy(sem_t *sem);
```

三、实验形式

个人实现

四、实验设备环境

openEuler 服务器和虚拟机。

五、实验步骤

5.1 读者-写者 实验原理及步骤

读者优先:

在 readFirst.c 中, 首先读取目标文件 test, 为每一行请求创建一个线程, 其中读请求创建读者线程, 写请求创建写者线程, 调用 pthread_create() 函数。引入计数器 reader_count 对读进程计数, writer_count 对写进程计数。mutex 是互斥信号量。

读者优先需要引入一个状态变量 state, 表明当前系统的状态。需要用到的几个状态量如下:

- mutex: 保证 readercount, writercount, state 变量访问的互斥性
- Sig_read: 允许读的信号量

Sig_wrt: 允许写的信号量

代码结构如下:

读者:

```
wait(mutex);
++reader_count;
if(state == s_waiting || state == s_reading) {
    signal(Sig_read);
    state = s_reading;
}
signal(mutex);
```

// read

```
wait(mutex);
--reader_count;
if(reader_count == 0) {
    if(writer_count != 0) {
        signal(Sig_wrt);
        state = s_writing;
    } else {
        state = s_waiting;
    }
}
signal(mutex);
```

写者:

```
wait(mutex);
```

```

++writer_count;

if(state == s_waiting) {

    signal(Sig_wrt);

    state = s_writing;
}

signal(mutex);

// write

wait(mutex);

--writer_count;

if(reader_count != 0) {

    signal(Sig_read);

    state = s_reading;
} else if(writer_count != 0) {

    signal(Sig_wrt);

    state = s_writing;
} else {

    state = s_waiting;
}

signal(mutex);

```

读者优先的设计思想是读进程只要看到有其它读进程正在读，就可以继续进行读；写进程必须等待所有读进程都不读时才能写，即使写进程可能比一些读进程更早提出申请。该算法只要还有一个读者在活动，就允许后续的读者进来，该策略的结果是，如果有一个稳定的读者流存在，那么这些读者将在到达后被允许进入。而写者就始终被挂起，直到没有读者为止。

写者优先：

在 writeFirst.c 中，首先读取目标文件 test，为每一行请求创建一个线程，其中读请求创建读者线程，写请求创建写者线程，调用 pthread_create() 函数。引入

计数器 `reader_count` 对读进程计数，`writer_count` 对写进程计数。 `mutex` 是互斥信号量。

写者优先需要引入一个状态变量 `state`，表明当前系统的状态。需要用到的几个状态量如下：

- `mutex`：保证 `readercount`, `writercount`, `state` 变量访问的互斥性
- `Sig_read`：允许读的信号量

`Sig_wrt`：允许写的信号量

代码结构如下：

读者：

```
wait(mutex);
++reader_count;
if(state == s_waiting
|| (state == s_reading && writer_count == 0)) {
    signal(Sig_read);
    state = s_reading;
}
signal(mutex);
```

// read

```
wait(mutex);
--reader_count;
if(writer_count != 0) {
    // 有读者在等待
    sem_post(Sig_wrt);
    state = s_writing;
} else if(reader_count == 0) {
    // 等待队列为空
    state = s_waiting;
}
signal(mutex);
```

写者：

```
wait(mutex);
++writer_count;
if(state == s_waiting) {
    signal(Sig_wrt);
    state = s_writing;
}
```

```

signal(mutex);

// write

wait(mutex);
--writer_count;
if(writer_count != 0) {
    signal(Sig_wrt);
    state = s_writing;
} else if(reader_count != 0) {
    for(int i=0; i!=reader_count; ++i) {
        signal(Sig_read);
    }
    state = s_reading;
} else {
    state = s_waiting;
}
signal(mutex);

```

写者优先的设计思想是在一个写者到达时如果有正在工作的读者，那么该写者只要等待正在工作的读者完成，而不必等候其后面到来的读者就可以进行写操作。该算法当一个写者在等待时，后到达的读者是在写者之后被挂起，而不是立即允许进入。

5.2 读者-写者 编译及运行方法

在装有 openEuler 操作系统的虚拟机或服务器上编译运行，注意需安装 GCC 编译器。

测试数据文件包括 n 行测试数据，分别描述创建的 n 个线程是读者还是写者，以及读写操作的开始时间和持续时间。

每行测试数据包括四个字段，各字段间用空格分隔。

第一字段为一个正整数，表示线程序号。

第二字段表示相应线程角色，R 表示读者是，W 表示写者。

第三字段为一个正数，表示读写操作的开始时间。线程创建后，延时相应时间（单位为秒）后发出对共享资源的读写申请。

第四字段为一个正数，表示读写操作的持续时间。当线程读写申请成功后，开始对共享资源的读写操作，该操作持续相应时间后结束，并释放共享资源。测试数

据 test 文件内容举例：

1 R 3 5

2 W 4 5

3 R 5 2

4 R 6 5

5 W 5.1 3

5.3 读者-写者 实验结果说明

程序运行结果分析首先需要从理论的角度分析，根据测试数据文件画出相应的读者优先和写者优先的线程运行顺序，如图 1。再将实际的实验运行结果与理论分析结果进行对比分析，以验证实验的正确性，并分析结果。

另外，可通过更改测试数据再次验证程序的正确性，同样需要先理论分析，再用实验结果与理论分析对比分析。

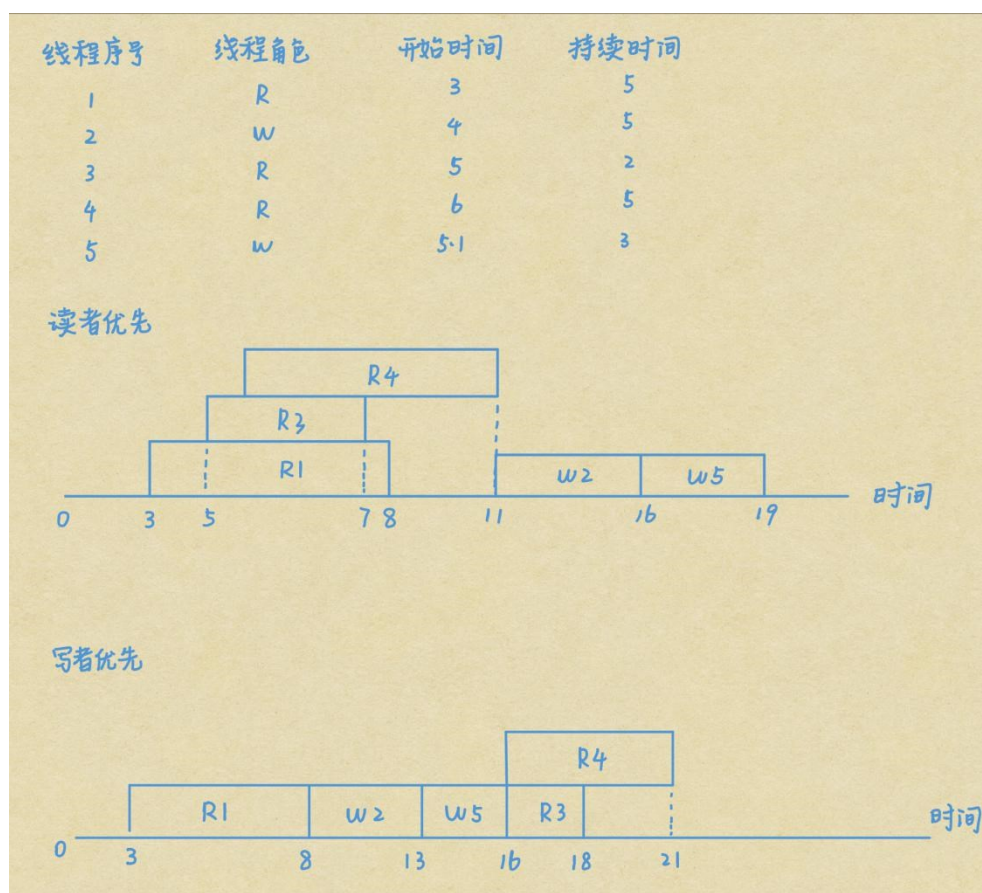


图 1 读者-写者实验测试数据理论分析图

5.4 生产者-消费者 实验原理及步骤

程序首先构造了一个生产者函数 `product()` 和一个消费者函数 `consumer()`。本程序有一个生产者和多个消费者。生产者每次在数组的一个单元里放入数字，下一次执行，在数组的下一个元素里填入数字。

消费者从数组里抓取数字。一个消费者抓取一个数字，下一个消费者抓取时，抓取下一个元素里的数字。

程序使用了一个互斥信号量 `mutex`，两个同步信号量 `full_sem` 和 `empty_sem`。使用 `mutex` 来保护整个生产和消费的过程。

对于生产者，当缓存充满时，等待 `empty_sem` 变量，等待消费者的消费；当完成填充时，激活 `full_sem` 变量，提示消费者现在可以抓取。

对于消费者，抓取之后，激活 `empty_sem` 变量，提示生产者可以开始工作；如果缓存是空的，就等待 `full_sem` 变量，等待生产者填入数字。

5.5 生产者-消费者 编译及运行方法

在 openEuler 虚拟机或服务器上编译运行即可。注意需安装 GCC 编译器。

5.6 生产者-消费者 实验结果说明

需要根据实验结果进行具体分析，自行可设计单生产者单消费者、多生产者多消费者、单生产者多消费者、多生产者单消费者多种模拟情况。

六、实验报告书写要求

学生的实验报告内容要与本指导书内容一致，要有：

实验目的：描述本次实验任务、目的；

实验要求及内容：描述本次实验具体要求，实验的具体内容，另外写出实验原理、整体思想，可给出相应原理图及整体框架图；

实验设备环境：描述本次实验的实验环境；

实验步骤（原理步骤、编译运行方法、实验结果说明）：首先写明实验的具体步骤，描述每个步骤对应的原理、流程图，写明实验的具体编译运行方法，最后展示实验结果，需要从理论分析和实际结果分析两方面分析对比实验结果，得出结论，会出现什么问题，如何解决；

实验心得体会：实验报告最后需要总结和写出心得体会，完成本次实验编写代码的时间、上机调试的时间、编程工具遇到的问题、编程语言遇到的问题、总结本次实验。